COSC 3500 Milestone 2

By Aryan Dhotre
s4985050

Forest Fire Cellular Automata

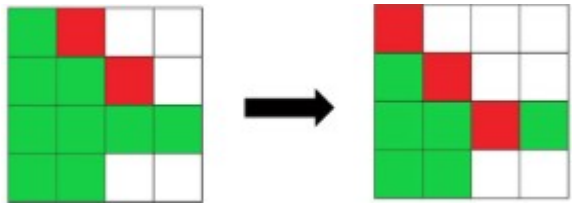


Fig.1. One timestep of forest fire model. Source: [1].

1. A tree grows on an empty cell with a probability p
2. A tree ignites with a probability f
3. A tree will burn if one of its neighbors is on fire
4. A burning cell turn into an empty cell

[1] “Forest-fire Model.” Accessed: Aug. 27, 2026. [Online]. Available: https://itp.uni-frankfurt.de/~gros/StudentProjects/Projects_2020/projekt_lars_dingeldein/
[2] B. Drossel and F. Schwabl, “Self-organized critical forest-fire model,” Phys. Rev. Lett., vol. 69, no. 11, pp. 1629–1632, Sep. 1992, doi: 10.1103/PhysRevLett.69.1629.

Benchmarking methodology

- git
- `time`
- tests at o3 , march=native
- tests for an 8032x8032 grid size for 500 timesteps
- each test ran 5 times, lowest score considered
- all tests have a warm up runs before the actual run
- slurm script prints the hardware configuration at the start of the job

data/v1.3/8-threads/slurm-logs/slurm-o3.out

```
1   ═══ execution environment ═══
2   opt flags: o3
3   job id:   689213
4   job node: a100-9
5   hostname: a100-9
6   kernel:   4.18.0-553.170.1.el8_10.x86_64
7   date:     2026-10-23T18:02:35+10:00
8   working directory: /home/Student/s4985050/forest-fire-sim
9   command: /var/slurm/d/job689213/slurm_script o3 v1.3
10  parameters: grids=8320 timesteps=500 warmups=0 reps=5 p=0.02 f=0.01
11  slurm allocation: job_cpus_per_node=8 cpus_per_task=8 ntasks=1
12  SLURM_CPUS_PER_TASK: 8
13  OpenMP: threads=8 dynamic=false places=cores proc_bind=close
14  process affinity: 0-7
15  pid 376699's current affinity list: 0-7
16  load average: 0.02 0.77 1.35 1/380 376712
17  uptime:  18:02:35 up 16 days, 22:39,  0 users,  load average: 0.02, 0.77, 1.35
18  Architecture:        x86_64
19  CPU(s):              8
20  Thread(s) per core:  1
21  Core(s) per socket:  8
22  Socket(s):           1
23  NUMA node(s):        1
24  Model name:          AMD EPYC 7542 32-Core Processor
25  Virtualization type: full
26  NUMA node0 CPU(s):   0-7
27  model name  : AMD EPYC 7542 32-Core Processor
28  pinned CPUs: 0-7
```

Program startup

Each binary prints which model configuration it's ran with , the version number and version name. It also identifies the gcc and c++ version

```
Model configuration
  Grid size:       8322
  Time steps:      500
  p (growth):      0.02
  f (lightning):   0.01
------------------------------------------------
Runtime information
  OpenMP threads:  8
------------------------------------------------
Build information
  Version          1.3 threads
  Compiled:        Oct 23 2026 17:11:40
  Compiler:        GCC 8.5.0 20210514 (Red Hat 8.5.0-28)
  C++ standard:    C++17 (__cplusplus=201703)
  Compiler target: x86_64-redhat-linux
  Executable:      bin/fire-v1.3-o3
  Build flags:     O3 NDEBUG -march=native -mavx2 -fopenmp
  Optimisation:    enabled
```

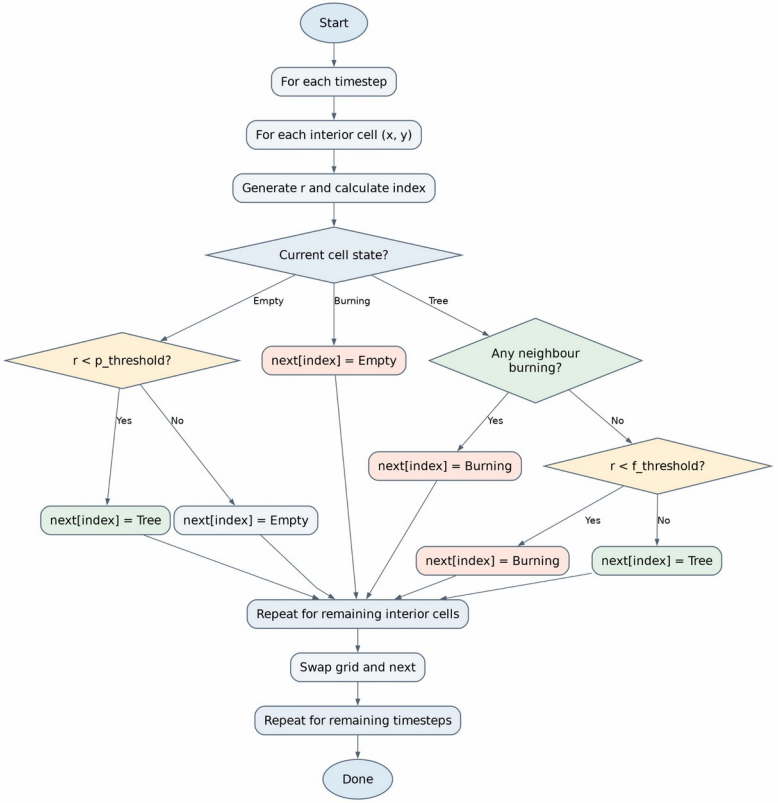
v1.0 - the serial baseline

- 117.77s to run an 8320x8320 grid for 500 timesteps
- compiled at o3 with march=native
- Double buffered

```
std::vector<CellState> grid(grid_size * grid_size);
std::vector<CellState> next(grid_size * grid_size);
pcg32 rng(80085);
for (int t = 0; t < timesteps; ++t) {
  for (int x = 1; x < grid_size - 1; ++x) {
    for (int y = 1; y < grid_size - 1; ++y) {
      auto r = rng.nextUInt();
      const auto index = x * grid_size + y;
      switch (grid[index]) {
      case CellState::Empty:…
      case CellState::Burning:
        next[index] = CellState::Empty;
        break;
      case CellState::Tree:
        const auto up = grid[index - 1];
        const auto down = grid[index + 1];
        const auto left = grid[index - grid_size];
        const auto right = grid[index + grid_size];
        if (up == CellState::Burning || down == CellState::Burning ||…} else {
          if (r < f_threshold) {…} else
            next[index] = CellState::Tree;
        }
        break;
      }
    }
  }
  std::swap(grid, next);
```

v1.1 - AVX

```
std::vector<CellState> grid(grid_size * grid_size);
std::vector<CellState> next(grid_size * grid_size);
pcg32 rng(80085);
for (int t = 0; t < timesteps; ++t) {
  for (int x = 1; x < grid_size - 1; ++x) {
    for (int y = 1; y < grid_size - 1; ++y) {
      auto r = rng.nextUInt();
      const auto index = x * grid_size + y;
      switch (grid[index]) {
      case CellState::Empty:…
      case CellState::Burning:
        next[index] = CellState::Empty;
        break;
      case CellState::Tree:
        const auto up = grid[index - 1];
        const auto down = grid[index + 1];
        const auto left = grid[index - grid_size];
        const auto right = grid[index + grid_size];
        if (up == CellState::Burning || down == CellState::Burning ||…} else {
          if (r < f_threshold) {…} else
            next[index] = CellState::Tree;
        }
        break;
      }
    }
  }
  std::swap(grid, next);
```



AI Disclosure

AI tools like chatgpt5.6-luna and claude-sonnet-5 were used to help with producing this assignment. I used ai tools to

- create the the scripts that visualised the simulation and benchmarks
- understand and analyze the benchmark data
- write slurm scripts to submit jobs
- polish my script for the presentation

Bibliography

[1] “Forest-fire Model.” Accessed: Aug. 27, 2026. [Online]. Available: https://itp.uni-frankfurt.de/~gros/StudentProjects/Projects_2020/projekt_lars_dingeldein/

[2] B. Drossel and F. Schwabl, “Self-organized critical forest-fire model,” Phys. Rev. Lett., vol. 69, no. 11, pp. 1629–1632, Sep. 1992, doi: 10.1103/PhysRevLett.69.1629.

[3] “The ‘Gru With a Gun’ Meme and the ‘Gru No’ Meme Come From a Doki Doki Literature Club Meme. : r/TheMatpatEffect.” Accessed: Aug. 27, 2026. [Online]. Available: https://www.reddit.com/r/TheMatpatEffect/comments/1rpku7c/the_gru_with_a_gun_meme_and_the_gru_no_meme_come/

[4] W. Jakob, wjakob/pcg32. (Jul. 23, 2026). C++. Accessed: Aug. 27, 2026. [Online]. Available: https://github.com/wjakob/pcg32